



4



13



1560 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

العمليات الأساسية على السجلات

د. سيرا أستور



RB Informatcs; 28/04/2025

بنیان الحاسوب ا

بسم الله الرحمن الرحيم

MIPS Machine Execution:

مقدمة:

يتم تنفيذ التعليمات بالتتالي line by line وتغيير القيم في السجلات والذاكرة.
ولكن قد ترى استثناءات لهذه القواعد:

Example:

المتغيرات	التعليمات
\$t0: 0, 12, 24, (?)	add \$t0, \$t1, \$t1
\$t1: 6, (?)	add \$t0, \$t0, \$t0
\$t2: 8	add \$t0, \$t0, \$t0
\$t3: 10	sub \$t1, \$t0, \$t1

شرح التعليمات:

نعلم أن السجل \$t0 يحتوي على القيمة 0 و \$t1 يحتوي على القيمة 6 و \$t2 يحتوي على قيمة 8، و \$t3 يحتوي على 10 قبل بدء التنفيذ.

نجمع قيمة \$t1 مع \$t1 ونضعها في \$t0: $6 + 6 = 12$

نجمع محتوى \$t0 مع نفسه مرتين: $12 + 12 = 24$

$24 + 24 = 48$

تعليمية sub:

نطرح \$t1 من \$t0 وتخزن في \$t1: $48 - 6 = 42$

انتبه:

\$s0 و \$t0 يختلفان عن السجل الصفري \$zero أو \$0.

MIPS Memory Organization:

تعتبر الذاكرة مصفوفة كبيرة أحادية البعد ولها عنوان.
عنوان الذاكرة: هو فهرس في المصفوفة نظراً لأن 8bit=byte مفيدة في العديد من البرامج فإن جميع البنى تقريباً اليوم تتعامل مع كل بايت على حدى.

Byte Addressing: means that the index points to a byte of memory

- بما أن الكلمة في MIPS هي 4 بايت فإن عنوانه البايت للذاكرة لها ضرب أربعة.
- العنوان هو عنوان أول بايت منه.

Example:

العنوان في الـ RAM	القيم المخزنة في الذاكرة
12	100
8	10
4	101
0	1

■ ملاحظات:

- كل الذاكر في الأسواق تعنون على مستوى Byte مثل: 4MB أو 4GB.
- الذاكر هي عبارة عن مواقع لا نخزن عنوانها أما العنوان هو index.

■ Alignment Restriction In MIPS

- يجب أن تبدأ الكلمات عند العناوين التي تكون مضروبة بـ 4.
- أي يجب محاذاة البيانات في الذاكرة على الحدود الطبيعية.
- ترتيب البايتات:

يمكن للمعالجات ترقيم البايتات داخل الكلمة بحيث يكون البايت الذي يحمل أقل رقم هو البايت الموجود في أقصى اليمين أو أقصى اليسار.

MIPS processors can operate with either big-endian or little-endian byte order.

بعض المعالجات تستعمل Little وبعضها Big.

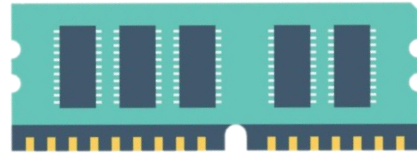
■ مثال:

Intel 80x86 is Little-endian, while Macintosh or Sun SPARC is big-endian.

	Byte #					Byte #			
عنوان	3	2	1	0		0	1	2	3
	Little Endian					Big Endian			

الحجم الكلي للعناوين:

- 2^{32} bytes with addresses from 0 to $2^{32}-1$
- 2^{32} words with addresses 0, 4, 8, ..., $2^{32}-4$



Word Addresses

16	32 bits of data
12	32 bits of data
8	32 bits of data
4	32 bits of data
0	32 bits of data

ملاحظات:

- الذاكرة تقوم بعنونة 0, 1, 2, ..., أما المعالج لا يقوم بعنونة كل سجلاتها بـ 32 بت، وعرض 32 Data Bus بت أيضاً.

$$2^{32} = 4 \text{ GByte}$$

لا يوجد اسئلة بالامتحان على هذه الفقرة ولكنها مهمة جدا لفهم باقي المحاضرات.

على الرغم من أن مساحة العنوان هي 32 بت إلا أن العناوين العليا من 0x80000000 إلى 0xFFFFFFFF غير متاحة لبرامج المستخدم.

تستخدم لنظام التشغيل وذاكرة القراءة فقط (ROM) عند استخدام شريحة MIPS في وحدة تحكم مدمجة فمن المحتمل أن يكون برنامج التحكم موجوداً في ذاكرة القراءة فقط (Rom) في هذا النصف العلوي من مساحة العنوان.

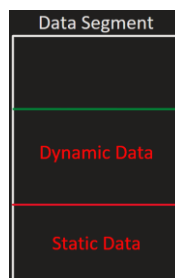
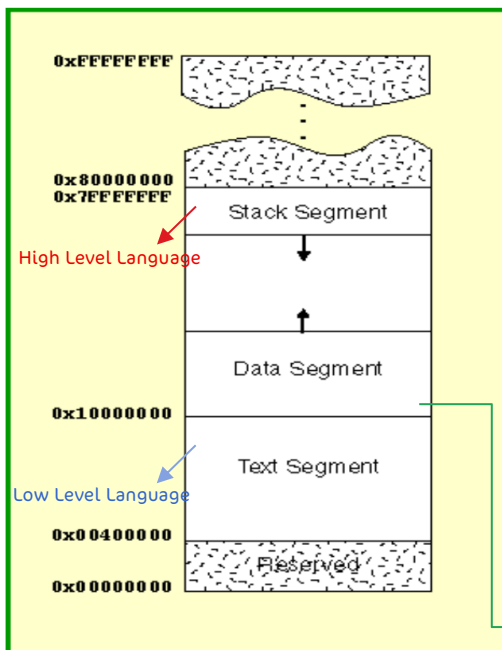
تنقسم أجزاء مساحة العنوان التي يمكن لبرنامج المستخدم الوصول إليها على النحو التالي:

- مقطع النص (text segment): يحتوي هذا المقطع على لغة الآلة لبرنامج المستخدم (الكود الذي سيتم تنفيذه).
- مقطع البيانات (Data segment): يحتوي هذا المقطع على البيانات التي يعمل عليها البرنامج، جزء من البيانات

static، هذه هي البيانات التي يتم تخصيصها بواسطة المجمع ولا يتغير حجمها أثناء تنفيذ البرنامج وفوق البيانات الثابتة توجد البيانات الديناميكية هذه هي البيانات التي يتم تخصيصها وإلغاء تخصيصها أثناء تنفيذ البرنامج.

مثلاً في لغة C يتم إجراء التخصيص وإلغاء التخصيص الديناميكي `malloc()` `free()`.

- مقطع المكس (stack segment): يوجد المكس في أعلى مساحة عنوان المستخدم في اللغات عالية المستوى، يتم دفع المتغيرات والمعاملات المحلية وإظهارها على المكس أثناء تنشيط الإجراءات وإلغاء تنشيطها.



MIPS Register Convention

Name	Register number	Usage
\$zero	0	the constant value 0
\$at	1	reserved for use by assembler
\$v0 - \$v1	2-3	values for results and expression evaluation
\$a0 - \$a3	4-7	arguments
\$t0 - \$t7	8-15	temporaries
\$s0 - \$s7	16-23	saved
\$t8 - \$t9	24-25	more temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	return address

Register 1 (\$at) reserved for assembler, 26-27 (k0-K1) for operating system.

الجدول السابق مهم جداً للحفظ.

■ بعض الملاحظات على الجدول:

1. يمكن أن تكتب Register number إما \$zero أو \$0، يستطيع محاكي MIPS فهم التعليمتين كما يمكن كتابة إما \$4 أو \$a0.
2. أكثر السجلات استخداماً \$t0, \$t9, \$s0, \$s7, كما يفضل عدم استخدام \$gp, \$sp, \$fp لأن لهم خصوصية محددة.
3. \$ra سجل مهم جداً ليعلم PC إلى أي تعليمة سوف يعود إليها بعد انتهاء تنفيذ البرنامج في الذاكرة.

■ توضيح:

عندما يهمل المعالج تنفيذ التعليمات بعد قراءتها تعليمة تعليمة في التابع يصل إلى تعليمة return ثم يعود إلى التابع الرئيسي main ويعلم مكان التعليمة التالية في الذاكرة عن طريق السجل \$ra لأنه يقوم بحفظ المكان قبل الانتقال إلى التابع.

ملاحظة:

يستخدم مع تعليمات J-type.



عمليات MIPS الحاسوبية:

التعليمات مثل السجلات وكلمات البيانات يبلغ طولها أيضاً 32bits.
تحتوي السجلات على الأرقام \$s1=17، \$s2=18، \$t1=9.

```
add $rd, $rs, $rt
```

```
add $t1, $s1, $s2
```

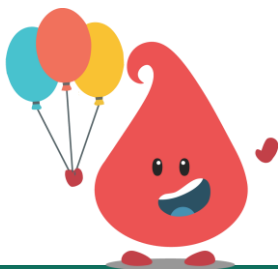
R-type: Register Instruction Format

000000	10001	10010	01001	00000	100000
op	rs	rt	rd	shamt	funct

- op: العملية الأساسية للتعليمية، وتسمى تقليدياً رمز التشغيل.
- rs: معامل مصدر السجل الأول.
- rt: معامل مصدر السجل الثاني.
- rd: معامل وجهة السجل يحصل على نتيجة العملية.
- shamt: (مقدار الإزاحة) يستخدم في تعليمات الإزاحة، ولن يستخدم حتى ذلك الحين، لذلك يحتوي الحقل على صفر في هذا القسم.
- funct: يحدد هذا الحقل الذي يسمى غالباً رمز الدالة المتغير المحدد للعملية في حقل العملية.

ملاحظة:

- أي حقل لا نستخدمه في التعليمية نضع فيه أصفار.
- op و funct تعطى كمعطيات.



Example:

All R-Type instructions have 3 operands, operand order is fixed (destination first)

C code	MIPS code
a=b+c	add a, b, c
a=b+c+d	add a, b, c add a, a, d

Signed and Unsigned Numbers

Number representations in binary

$$(z_{31} \times 2^{31}) + (z_{30} \times 2^{30}) + (z_{29} \times 2^{29}) \dots \dots \dots (z_1 \times 2^1) + (z_0 \times 2^0)$$

$$1011_{two} = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) - (1 \times 2^0)_{ten} = (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1)_{ten} = 8 + 0 + 2 + 1_{ten} = 11_{ten}$$

وترقيم البيئات داخل كلمة MIPS وموضع الرقم 11_{ten} كما في الجدول:

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	1 0 1 1
عرض 32 بت							

البت الأقل أهمية LSB أقصى بت يمين في كلمة MIPS.

البت الأكثر أهمية MSB أقصى بت يسار في كلمة MIPS.

1. Positive & Negative Representation:

تمثيل الإشارة والمقدار: عن طريق إضافة إشارة منفصلة والتي يمكن تمثيلها بسهولة في Bit واحد.

$$+7_{ten} = 0111_{two}$$

$$-7_{ten} = 1111_{two}$$

Several short comings:

أين نضع بت الإشارة؟! بت الإشارة المنفصل سيجعل لدينا صفر موجب وسالب.

$$+0_{ten} = 0000_{two}$$

$$-0_{ten} = 1000_{two}$$

2. one's Complement (مكمل العدد الأحادي):

يمثل أصغر قيمة سالبة بمقدار $100\dots00_{two}$ وأكبر قيمة موجبة بمقدار $01\dots11_{two}$. تاركاً عدداً متساوياً من الأعداد السالبة والموجبة ولكن في النهاية بصفرين أحدهما موجب ($000\dots000_{two}$) والآخر سالب ($111\dots111_{two}$).

$$+7_{ten} = 0111_{two}$$

$$-7_{ten} = 1111_{two}$$

3. Solution? Pick the representation that made the hardware simple:

Leading 0s mean positive, and leading 1s mean negative.

It is called Two's Complement Representation (يدعى المتمم الثنائي).

يتميز بأن جميع الأعداد السالبة تحتوي على (1) في البت الأكثر أهمية.

تحتاج الأجهزة إلى اختبار هذا البت فقط لمعرفة ما إذا كان الرقم موجباً أم سالباً (مع اعتبار الرقم 0 موجباً) غالباً ما يطلق على هذا البت اسم بت الإشارة (Sign bit).

من خلال التعرف على دور بيت الإشارة يمكننا تمثيل الأعداد الموجبة والسالبة المكونة من 32 بت.

$$(z_{31} \times 2^{31}) + (z_{30} \times 2^{30}) + (z_{29} \times 2^{29}) \dots (z_1 \times 2^1) + (z_0 \times 2^0)$$

■ Example: convert to decimal:

1111 1111 1111 1111 1111 1111 1100_{two}

■ Example:

convert the 16 bits representation of 2 into 32 bits representation of -2

-2 _{ten}	0000	0000	0000	0010
	1111	1111	1111	1101
	+			1
	1111	1111	1111	1110

creating 32-bit version: من العدد السالب يعني نسخ بت الإشارة 16 مرة ووضعه على اليسار

$$-2_{ten} = 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two}$$

MIPS Addition Operations

ADD VS ADDU:

أراد مصممو عمليات MIPS التأكد من أن نتائج التعليمات تكون دائماً "صحيحة" وفقاً لمواصفاتهم.

الرغبة في الصحة في قيود التعارض للتمثيل المحدود، خاصة في حالة بعض العمليات الحسابية.

مثال: قد ينتج عن إضافة رقمين من 32 بت نتيجة 33 بت، أو الأسوأ من ذلك عند استخدام تمثيل (2ⁿ-complement)

قد ينتج عن إضافة رقمين موجبين نتيجة سالبة تسمى هذه الشذوذات "التدفقات الزائدة" (overflows).

Two ways to fix this:

■ إجراء اختبار صريح إما قبل أو بعد كل عملية (تكلفة إضافية باهظة).

■ إنشاء "استثناء" (Exception) في حالة حدوث (overflows).

○ ADD: ينشئ استثناءات عند حدوث overflow.

○ ADDU: لا ينشئ استثناءات عند حدوث overflow.

MIPS Logical Operations – Shift الإزاحة

SLL: Shift Logical Left (الإزاحة المنطقية لليسار):

sll rd, rt, shamt

الصيغة العامة:

	000000	00000	00010	00010	00100	00000001
R-type	op=0x00 dictating an ALU function	Unused set to "0"	rt=2 Reg[2] source	rd=2 Reg[2] destination	Shamt=4 Dictates a shift of 4- bits	Func=0x00 dictating an sll

توضيح للجدول:

SLL \$2, \$2, 4 → مقدار الإزاحة

#\$2=\$2<<4

شرح التعليمة: نقوم بإزاحة السجل الثاني بمقدار 4 خانات، هذا يعادل:\$2=\$2*2⁴الإزاحة من اليسار تعادل ضرب القيمة الموجودة بالسجل بـ 2^(مقدار الإزاحة)

تعليمات الإزاحة في MIPS:

1. SLL (Shift Left Logical): "إزاحة منطقية لليسار" إزاحة لليسار وملء البتات الفارغة بـ 0.
2. SRL (Shift Right Logical): "إزاحة منطقية لليمين" إزاحة لليمين وملء البتات الفارغة بـ 0.
3. SRA (Shift right arithmetic): إزاحة لليمين وملء البتات الفارغة بامتداد الإشارة (sign extending).

Example:

shift right by 8 arithmetic:

1001	0010	0011	0100	0101	0110	0111	1000
1111	1111	1001	0010	0011	0100	0101	0110

قمنا بإتمام واحدات لأن آخر خانة (خانة الإشارة) واحد (1)، للحفاظ على العدد السالب.

Masking الإخفاء:

1011	0110	1010	0100	0011	1101	1001	1010
0000	0000	0000	0000	0000	1111	1111	1111

نتيجة AND لهذين العنصرين:

0000	0000	0000	0000	0000	1101	1001	1010
------	------	------	------	------	------	------	------

يستخدم تعليمة andi , and لعزل البتات والبايتات في كلمة.

- Zeroing some bits in a word.
Example: `andi $t0, $t0, 0xFFFF`
- Setting some bits as ones in a word.

يستخدم تعليمة ori , or لتعيين بعض البتات كواحدات في الكلمة.

Let t0=0X12345678

Then after: `ori $t0, $t0, 0xFFFF`

\$t0=0x1234FFFF

0xFFFF = 1111 1111 1111 1111

0x 5678 = 1000 0111 0110 0101

1000 0111 1111 1111

لا يتم التعديل بـ 16 بت عالية الترتيب بينما يتم التعديل بـ 16 بت منخفضة الترتيب على 1s.

Multiplication (الضرب)

In MIPS:

32-bits Value × 32-bits Value = 64-bit Value

General Syntax:

`mult register1, register2`

تخزين 32 بت high في السجل HI.

تخزين 32 بت low في السجل LO.

نتعامل معه على أنه ناتج الضرب إذا كانت الأعداد صغيرة نسبياً أي أصغر من 32 بت.

Example:

In C:	In MIPS:	comments:
<code>a=b*c</code>	<code>mult \$s2, \$s3</code>	<code>#b*c</code>
	<code>mfhi \$s0</code>	<code>#upper half of product into \$s0</code>
	<code>mflo \$s0</code>	<code>#lower half of product into \$s1</code>

Division (القسمة)

General Syntax:

```
div register1, register2
```

- يقسم قيم 32 بت في السجل (1) على قيمة 32 بت في السجل (2).
- يضع remainder (باقي القسمة) في HI.
- يضع quotient (حاصل القسمة) في LO.

ملاحظة: يمكننا استخدام هذه التعليمات لبرمجة / و % بلغة "C".

MIPS Immediate ALU Operations

مثال: -3, \$11, \$9 add

	001000	01011	01001	111111111111 01
I-type:	OP = 0x08 dictating addi	rs=11, Reg[11] source	rt=9, Reg[9] destination	تعمل مثل الثوابت Constant Field indicating -3 as second Operand (sign-extended)

General Syntax:

```
addi rt, rs immediate
```

Example: addi \$21, \$22, -50

ترجمة عملية الـ MIPS ALU إلى لغة الآلة.

- opcode (رمز التعليم) = 8 (من الجدول).
- rs = 22 (السجل الذي يحتوي على المعامل).
- rt = 21 (السجل المستهدف).
- immediate = -50 (افتراضياً هذا عدد عشري).

التمثيل العشري لكود الآلة	8	22	21	-50
التمثيل الثنائي لكود الآلة	001000	10110	10101	111111111001110

الثوابت الكبيرة Large constants

لتحميل ثابت 32 بت في سجل يتم استخدام تسلسل تعليميتين:

Load Upper immediate:

`lui $8, 1010101010101010`

ثم يجب الحصول على البتات ذات الترتيب الأدنى بشكل صحيح أي:

`ori $8, $8, 1010101010101010` (جمع منطقي or)

ملاحظة:

العنوان في الذاكرة يكون مخزن كعنوان قاعدي في rs والـ imm يمثل مقدار من مضاعفات 4 للانتقال إلى العناوين المجاورة (نسبيه offset).

ori	1010101010101010	0000000000000000
	0000000000000000	1010101010101010
	1010101010101010	1010101010101010



Just a little bit!

MIPS Load/Store Instructions

General Syntax:

1. `lw rt, immediate (rs)` # $\text{Reg}[rt] = \text{Mem}[\text{Reg}[rs] + \text{sxt}(\text{imm})]$

جلب محتويات موقع الذاكرة الذي يكون عنوانه const immediate (فوري) إلى rt.

Abbreviation:

`Lw rt, imm` for `Lw rt, imm($0)`

2. `sw rt, immediate (rs)` # $\text{Mem}[\text{Reg}[rs] + \text{sxt}(\text{imm})] = \text{Reg}(rt)$

تخزين محتويات rt في موقع الذاكرة الذي يكون عنوانه const.

Abbreviation:

`sw rt, imm` for `sw rt, imm($0)`

عناوين البايت ولكن sw, lw 32 بت للوصول إلى عناوين الكلمات المحاذية يجب أن يكون أقل بيتين للعناوين الناتجين 0.

■ MIPS هي بنية تحميل وتخزين

يعني أن جميع عمليات الوصول إلى ذاكرة البيانات تقتصر على تعليمات التخزين والتحميل (التي تنقل محتويات السجل من وإلى الذاكرة، وهو سبب تحميل البيانات من الذاكرة للمعالج بدلاً من التعديل عليها في الذاكرة مباشرة).

■ Example:

For binary Representation: `lw $t0, 32($s2)`

35	18	8	32	
op	rs	rt	16 bit number	I-type instructions

ستأخذ هذه التعليمات المؤشر في `$s2` (السجل القاعدي) وتضيف 32 بايت (إزاحة) إليه، ثم تحمل القيمة من الذاكرة التي يشير إليها هذا المجموع المحسوب في السجل `$t0`، أي من الـ RAM إلى الـ register في المعالج.

Translating MIPS Assembly Language into Machine Language

Example:

If `$t1` has the base of the array `A` and `$s2` corresponds to `h`, the assignment statement:

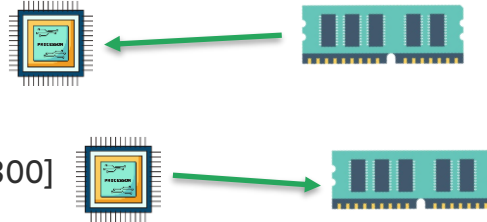
`A[300] = h + A[300];`

is compiled into:

`lw $t0, 1200($t1)` # reg `$t0` gets `A[300]`

`add $t0, $s2, $t0` # reg `t0` gets `h + A[300]`.

`sw $t0, 1200($t1)` # stores `h + A[300]` back into `A[300]`



ملاحظة أرييسزية:

كل عنصر في مصفوفة `int` حجمه 4 byte.

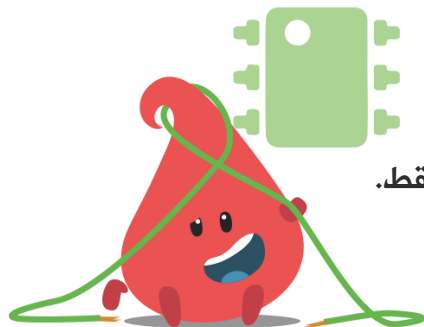
يسمى الثابت في تعليمة نقل البيانات (300) الإزاحة (offset) ويسمى السجل المضاف لتشكيل العنوان (`$t1`) السجل القاعدي.

المصفوفة `A` مخزنة في الـ ram في عنوان ما (0x524.....)، قمنا بشحن السجل `$t1` بهذا العنوان (pointer) قبل بدء التنفيذ.

في تعليمة `lw` جلبنا محتويات العنوان الموجود في `$t1` مع الإزاحة 1200 بايت (لأن المتحولات `int` حجمها 4 بايت) إلى السجل `$t0` (أي أصبح هناك نسخة من هذه البيانات في السجل `$t0`).

قمنا بعملية الجمع، ثم خزنا القيمة الجديدة في العنوان نفسه (أي أن `$t1` يحتوي على العنوان فقط وليس ببيانات المصفوفة).

قمنا بالتحميل من الـ RAM إلى الـ register لأن العمليات الحسابية تنفذ في السجلات فقط.



ملخص بسيط:

Instruction	Format	Op
add	R	0
sub (subtract)	R	0
add immediate	I	8 _{ten}
lw (load word)	I	35 _{ten}
sw (store word)	I	43 _{ten}

Example:

باستخراج العملية المعطاة وعدد السجلات المستخدمة ما هو رمز لغة الآلة MIPS لهذه التعليمات؟

\$t0=8, \$t1=9, \$s1=18

op	rs	rt	rd	shamt	funct
35	9	8	1200		
0	18	8	8	0	32
43	9	8	1200		

Binary Equivalent:

10011	01001	01000	0000010010110000		
000000	10010	01000	01000	00000	100000
101011	01001	01000	0000010010110000		

انتهت المحاضرة!

بالتوفيق ♥